[10 marks] Question 1: Convexity and Optimization

[4 marks] Q1.1: Prove the given function is convex:

$$||y - X\theta||_2^2 \tag{1}$$

Where $y \in R^{mx1}$, $X \in R^{mxn}$ and $\theta \in R^{nx1}$

[6 marks] Q1.2: Considering the previous convex function, solve the optimization given the constraints:

$$\begin{aligned} &\min_{\theta} ||y - X\theta||_2^2 \\ &\text{s.t. } \omega^T\theta = b \end{aligned} \tag{2}$$

where additionally $\omega \in R^{nx1}$ and b is constant.

[20 marks] Question 2: The Curse of Dimensionality

As we go into higher-dimension spaces, there is very little difference between the distances of almost all pairs of points in the space. This is popularly referred to as "The Curse of Dimensionality".

[3 marks] Q2.1: Consider two independent univariate random variables X and Y sampled uniformly from the unit interval [0, 1]. Determine the expectation and variance of the random variable $Z = |X - Y|^2$, i.e., the squared distance between X and Y.

[4 marks] Q2.2: For each choice of dimension $d \in [2^0, 2^1, 2^2, ..., 2^{10}]$, sample 100 points from the unit cube, and record the following average distances between all pairs of points, as well as the standard deviation of the distances.

1. Squared Euclidean (or squared ℓ_2) distance $= ||x - y||_2^2 = (X - Y)^2$
2. Manhattan Distance (or ℓ_1) distance $= ||x - y||_1 = |X - Y|$

Plot both the average and standard deviation of pairwise distances as a function of d.

[4 marks] Q2.3: In this question, we aim to verify our simulations in part II. by deriving the analytical form of mean and variance of squared Euclidean distance in $\mathbb{R}$. Suppose we sample two points X and Y independently from a unit cube in d dimensions (i.e., $X, Y \in \mathbb{R}^d$). Define the squared Euclidean distance between X, Y as $R = Z_1 + ... + Z_d$ with $Z_i = (X_i - Y_i)^2$. Given the mean and variance from part I (for random variables in $\mathbb{R}^1$), determine E[R] and Var[R] using the properties of expectation and variance. You may give your answer in terms of the dimension d.

[6 marks] Q2.4: In probability theory, we can use Markov's Inequality to derive

$$P\Big(|Z - E[Z]| \geq a\Big) \leq \frac{Var[Z]}{a^2} \tag{3}$$

for any random variable Z and constant a. In this part, use Equation (3) to prove the claim that in high dimensions (i.e., as $d \to \infty$), most points become approximately the same distance (i.e., by showing that likelihood of R being far from its expectation becomes really small).

[2 marks] Q2.5: Consider the unit cube in d-dimensional space (i.e., $[0, 1]^d$) containing n training samples sampled uniformly within this cube. Further consider a test point $\hat{x}$ and k of its nearest neighbors from the training set. Let l be the edge length of the smallest hyper-cube (i.e., an n-dimensional analog to a square) within this unit cube that contains all k nearest neighbors of the test point. It can be shown that $l^d \approx k/n$. For $k = 10$ and $n = 1000$, calculate the approximate value of l for varying $d = 10^{\{0,1,2,3,4\}}$. Do you notice any issues that may arise, and what does the pattern mean?

[1 mark] Q2.6: One solution to this issue would be to increase the number of samples. Explain why this solution is not practical.

[10 marks] Question 3: Nearest Neighbors: Do NOT use scikit-learn unless specified otherwise.

[5 marks] Q3.1: A feature selection analysis on the Pima Indians Diabetes Database identifies three factors, i.e., Plasma glucose concentration 2 hours in an oral glucose tolerance test, Body mass index (weight in $kg/(\text{height in m})^2$), and Age (years) as being highly relevant factors for predicting whether or not a person in the database has diabetes. In this exercise, you will implement a KNN using the distance metrics below, and evaluate them on a simplified version of the Pima dataset found here. Grades will be assigned to the code implementation of your KNN, as well as to comparisons done with sklearn's implementation thereof.

1. Implement a KNN that takes as input the number of neighbors k and metric $= \{\text{Euclidean}, \text{Cosine}\}$ as well as a training set and validation set, and measures performance on the validation set and test set. (3 marks)

2. Implement the Euclidean Distance for vectors $x, y \in \mathbb{R}^d$. (1 mark)

$$d(x, y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2} \tag{4}$$

3. Implement the Cosine Similarity for vectors $x, y \in \mathbb{R}^d$. (1 mark)

$$cos(\theta) = \frac{\langle a, b \rangle}{||a||_2 . ||b||_2} \tag{5}$$

[2 marks] Q3.2: Split the dataset into the following train/val/test ratios and train a 1-Nearest Neighbor model using both your implementation and sklearn's implementation on the two metrics above. Make sure to use the same data splits when training the two implementations. Report the validation errors for your implementation vs that of the sklearn's across all settings. (2 marks)

1. 80/10/10 (0.5 marks)

2. 34/33/33 (0.5 marks)

3. 25/25/50 (0.5 marks)

4. Which combination works best, and why? (0.5 marks)

[3 marks] Q3.3: Compare the two metrics from part I, three variations of train/val/test splits from part II, and additionally with $k = 1, 3, 5, 11$. Which combination is the best? Explain in terms of train/val/test errors and support with appropriate graphs.

[20 marks] Question 4: Decision Trees Do NOT use scikit-learn unless specified otherwise.

The Titanic dataset from Kaggle is one of the most popular beginner datasets available on the internet. The dataset aims to predict whether a passenger aboard the original 1912 shipwreck survived the iceberg or not, given a list of information about each passenger (e.g. their name, age, gender, passenger class, fare, etc). A decision tree with only one split is called a *decision stump*. A split partitions the passengers into two groups using a feature and a rule, such as a gender category or an age threshold. Each resulting leaf predicts a class label for the passengers in that group.

We prefer splits that produce groups with less mixed survival labels. A group in which everyone has the same label is *pure*. We will first evaluate a tree by counting the mistakes it makes when each leaf predicts its majority class. Dividing this count by the total number of passengers gives the tree's error rate on the supplied data. Later, we will introduce Gini impurity and entropy as alternative measures of label mixing, and evaluate predictive performance on held-out data.

Using a simplified version of the original dataset at this link, answer the questions below.

[1 mark] Q4.1: Split the passengers into two groups using the dataset's `Gender` variable (0 = female, 1 = male). Use all rows of the supplied simplified dataset; `Survived` is 1 for survivors and 0 for non-survivors.

1. For each group, report the number of survivors and non-survivors. Suppose the tree predicts the majority class within each group. State the prediction for each group and how many passengers it misclassifies.

2. What fraction of all passengers would this tree misclassify? Show your calculation.

This fraction is the tree's classification error rate on the supplied data. No train/test split or decision-tree library is needed here.

[1 mark] Q4.2: Impurity becomes more important when the decision tree is deciding on a split between a *numerical* variable as opposed to a *categorical* variable. Why?

[2 marks] Q4.3: Consider a tree with two successive splits along each path (three levels including the root and leaves). First split on `Gender`, then split each group using the rule `Age` ≤ 65 versus `Age` > 65.

1. Report the survivor and non-survivor counts in each final group. Predict the majority class in each nonempty group, and compute the tree's total error count and error rate. Does the additional age split reduce the errors compared with Q4.1?

2. Reverse the order: first split on age using the same threshold, then split each resulting group on gender. Do the final groups or the total error count change? Explain.

Use all rows of the supplied dataset. An empty group contributes zero errors; no prediction needs to be assigned to it for this calculation.

From error counts to impurity scores. For a nonempty node t, let $p_{t,c}$ be the fraction of its passengers with survival label $c \in \{0, 1\}$. Classification-error impurity is $I_{\text{err}}(t) = 1 - \max_c p_{t,c}$. Gini impurity and entropy below also use these *label proportions*, rather than the distribution of the input feature. To combine node scores, weight them by group size:

$$I_{\text{tree}} = \sum_{\text{nonempty leaves } t} \frac{n_t}{N} I(t),$$

where n_t is the number of passengers in leaf t and N is the total. For a single split, the leaves are its two children. With classification-error impurity, $n_t I_{\text{err}}(t)$ is the error count in that leaf, so this formula gives the same total error rate as before. Counts are added directly; the weights are needed when combining *rates or scores*.

[2 marks] Q4.4: Decision trees typically choose splits greedily, comparing candidate splits at the current node rather than searching over complete trees. One criterion is Gini impurity:

$$G(t) = 1 - \sum_{c \in \{0,1\}} p_{t,c}^2.$$

For binary labels, it ranges from 0 (pure) to 1/2 (equally mixed). Implement a function that computes Gini impurity from the labels in a node. Use the weighted score above to compare two candidate root splits: gender, and `Age` ≤ 65 versus `Age` > 65. Which split has lower weighted Gini impurity? Use all rows, as in the preceding parts.

[2 marks] Q4.5: Alternatively, Shannon entropy measures label mixing as

$$H(t) = - \sum_{c \in \{0,1\}} p_{t,c} \log_2 p_{t,c},$$

with $0 \log_2 0 = 0$. For binary labels it ranges from 0 (pure) to 1 (equally mixed). Implement a function that computes this entropy from the labels in a node. Compare the weighted entropy scores of the same gender and age splits. Does entropy prefer the same split as Gini impurity?

[3 marks] Q4.6: When building a model using sklearn, it is important to look at the documentation to understand how to adjust its performance. The decision tree documentation includes several different parameters that can be tweaked around (intelligently) to improve the resultant decision tree.

1. Using sklearn.tree.DecisionTreeClassifier, train a decision tree classifier on the entire dataset (no val or test sets in this part) using "Survived" as the target feature. Set the max_depth=3 and random_state=123123 and keep other parameters as default values. Plot the tree using the tree.plot_tree function to visualize the resultant tree. Which two features are used to split the decision at depth=2? (1 mark)

2. Do the features at depth=3 change if you set criterion="entropy" instead of the default? (1 mark)

3. Do the features at depth=3 change if you set splitter="random"? (1 marks)

[6 marks] Q4.7: Randomly changing parameters doesn't tell us much about which decision trees are better than others. We need something to compare them against each other (and potentially other models). This is where the validation and test sets comes in. Split the dataset into the following train/val/test ratios and compare the accuracy scores of the resulting decision trees for max_depth=3 and random_state=123123 and criterion={gini, entropy}.

1. 80/10/10

2. 34/33/33

3. 25/25/50

[3 marks] Q4.8: How does the model change as you change the max_depth? Are there tradeoffs to consider?